Department of Computer Science and Engineering

Jashore University of Science and Technology

Faculty of Engineering and Technology - FET

Lab Report

On

CNN Image Classification with PyTorch

Date of submission: 17 August 2026

Submitted To:

Instructor: Dr. A F M Shahab Uddin

Assistant Professor

Department of CSE, JUST

Course Title: Artificial Intelligence and Machine Learning Lab

Course Code: CSE 3202

Submitted By:

Name: Shafikul Islam Marwan

Student ID: 220121

Registration Number: 2201020

Year: 3rd

Semester: 2nd

Contents

1 Introduction

Convolutional Neural Networks (CNNs) are the standard deep learning architecture for image classification tasks because they exploit spatial locality and translation invariance through learned convolutional filters. This lab implements a complete CNN classification pipeline in PyTorch that bridges standard benchmark-dataset training with real-world testing, as required by the assignment brief (*Assignment: CNN Image Classification with PyTorch*).

The chosen task is **Option 2: Hand Gestures (Rock–Paper–Scissors)**. A CNN is trained from scratch on the publicly available Rock–Paper–Scissors (RPS) dataset created by Laurence Moroney, and is then evaluated on 10 custom photographs of the author's own hand captured with a smartphone camera, showing the Rock, Paper, and Scissors gestures.

1.1 Objectives

- Build a full CNN training/evaluation pipeline in PyTorch, automated end-to-end in Google Colab.
- Automatically retrieve the standard RPS dataset (via direct download) and custom phone images (via `git clone` of a GitHub repository) with no manual uploads.
- Design, train, and evaluate a CNN classifier on the standard dataset.
- Test the trained model on real-world smartphone photographs and critically analyze the results.
- Visualize training curves, a confusion matrix, misclassified samples, and a custom-image prediction gallery.

2 Dataset Description

2.1 Standard Dataset – Rock, Paper, Scissors

The model is trained on the well-known synthetic Rock–Paper–Scissors dataset released by Laurence Moroney, downloaded automatically inside the notebook from:

- Training set: https://storage.googleapis.com/download.tensorflow.org/data/rps.zip
- Test set: https://storage.googleapis.com/download.tensorflow.org/data/rps-test-set.zip

The dataset contains CGI-rendered images of hands making the three gestures against a plain background, at varying angles, skin tones, and lighting. The class distribution is perfectly balanced, as shown in Table 1.

Table 1: Rock–Paper–Scissors dataset split summary

Split	Paper	Rock	Scissors
Training	840	840	840
Test	124	124	124
Total	964	964	964

In total there are **2,520 training images** and **372 test images** across the three balanced classes: `paper`, `rock`, `scissors`.

Figure 1: Sample images from the RPS training set, one batch drawn at random with augmentation applied.

2.2 Custom Real-World Dataset

To evaluate real-world generalization, 10 original smartphone photographs were taken of a human hand performing the three gestures against a plain wall background: 3 images of `paper`, 4 images of `rock`, and 3 images of `scissors`. These images were committed to the `dataset/` folder of the GitHub repository and are cloned automatically at runtime – no manual upload is required.

3 Methodology

3.1 Pipeline Overview

The entire workflow is automated inside a single Colab notebook (`220121.ipynb`) and executes without any manual intervention, satisfying the assignment's reproducibility requirement:

1. Clone the GitHub repository with `!git clone` to obtain the 10 custom phone images.
2. Download and extract the standard RPS train/test archives with `requests` and `zipfile`.
3. Build `torchvision.transforms` pipelines and `DataLoader` objects.
4. Define the CNN architecture as an `nn.Module` subclass.
5. Train for 10 epochs, tracking loss/accuracy history.
6. Save the trained weights as `220121.pth` (both locally and into the cloned repo's `model/` folder).
7. Generate training curves, a confusion matrix, and visual error analysis on the standard test set.
8. Run inference on the 10 custom phone images and display a prediction gallery.

3.2 Data Preprocessing

All images are resized to 150×150 pixels and normalized using ImageNet mean/std statistics, so that the pretrained-style normalization is consistent across both the standard dataset and the custom phone photos. For the training split, additional data augmentation is applied to improve generalization; the test/validation split uses only deterministic resizing and normalization.

Table 2: Preprocessing / augmentation pipeline

Stage	**Transform**
Training	Resize(150,150) RandomHorizontalFlip($p = 0.5$) RandomRotation($\pm 10°$) ColorJitter(brightness=0.1, contrast=0.1) ToTensor() Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
Test / Custom Images	Resize(150,150) ToTensor() Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

Datasets are loaded with `torchvision.datasets.ImageFolder` and wrapped in `DataLoader` objects with `batch_size=64`.

3.3 CNN Model Architecture

A CNN was built from scratch (no pretrained backbone) consisting of three convolutional blocks followed by a fully connected classifier head. Each convolutional block uses a 3×3 convolution, batch normalization, ReLU activation, and 2×2 max pooling. The classifier head uses adaptive average pooling to a fixed 4×4 spatial size (making the network robust to the 150×150 input size), followed by two fully connected layers with dropout for regularization.

Table 3: CNN architecture summary

Block	**Layers**	**Output Channels**
Conv Block 1	Conv2d(3,32,k=3,p=1) → BatchNorm2d → ReLU → MaxPool2d(2,2)	32
Conv Block 2	Conv2d(32,64,k=3,p=1) → BatchNorm2d → ReLU → MaxPool2d(2,2)	64
Conv Block 3	Conv2d(64,128,k=3,p=1) → BatchNorm2d → ReLU → MaxPool2d(2,2)	128
Classifier	AdaptiveAvgPool2d(4,4) → Flatten	128×4×4 = 2048
	Dropout(0.4) → Linear(2048, 256) → ReLU	256
	Dropout(0.3) → Linear(256, 3)	3

Total parameters: 619,011 (all trainable).

```
class CNN(nn.Module):
    def __init__(self, num_classes=3):
        super(CNN, self).__init__()

        # first conv block
        self.conv1 = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
        # second conv block
        self.conv2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
```

```
            nn.MaxPool2d(2, 2)
        )
        # third conv block
        self.conv3 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
        # classifier head
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((4, 4)),
            nn.Flatten(),
            nn.Dropout(0.4),
            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = self.conv3(x)
        x = self.classifier(x)
        return x
```

Listing 1: CNN model definition in PyTorch

3.4 Training Configuration

Table 4: Training hyperparameters

Hyperparameter	**Value**
Loss function	CrossEntropyLoss
Optimizer	Adam
Learning rate	0.001
Batch size	64
Epochs	10
Input image size	$150 \times 150 \times 3$
Random seed	42

4 Results

4.1 Training History

The model was trained for 10 epochs. Training loss decreased steadily and training accuracy exceeded 99% by epoch 6, while validation loss/accuracy fluctuated across epochs (Table 5, Figure 2). The instability in validation metrics around epochs 3 and 7 suggests the model was somewhat sensitive to the batch statistics and learning rate at those points, though it recovered in subsequent epochs and reached its best validation accuracy (95.43%) at the final epoch.

Table 5: Per-epoch training and validation metrics

Epoch	Train Loss	Train Acc.	Val. Loss	Val. Acc.
1	0.6223	0.7460	0.3053	0.8333
2	0.1468	0.9587	0.1956	0.9382
3	0.0957	0.9762	1.1049	0.6855
4	0.0833	0.9734	0.3243	0.9140
5	0.0800	0.9770	0.4889	0.8280
6	0.0360	0.9901	0.1203	0.9435
7	0.0241	0.9937	1.0759	0.7849
8	0.0200	0.9937	0.2145	0.9301
9	0.0235	0.9917	0.2970	0.9274
10	0.0261	0.9913	0.1396	**0.9543**

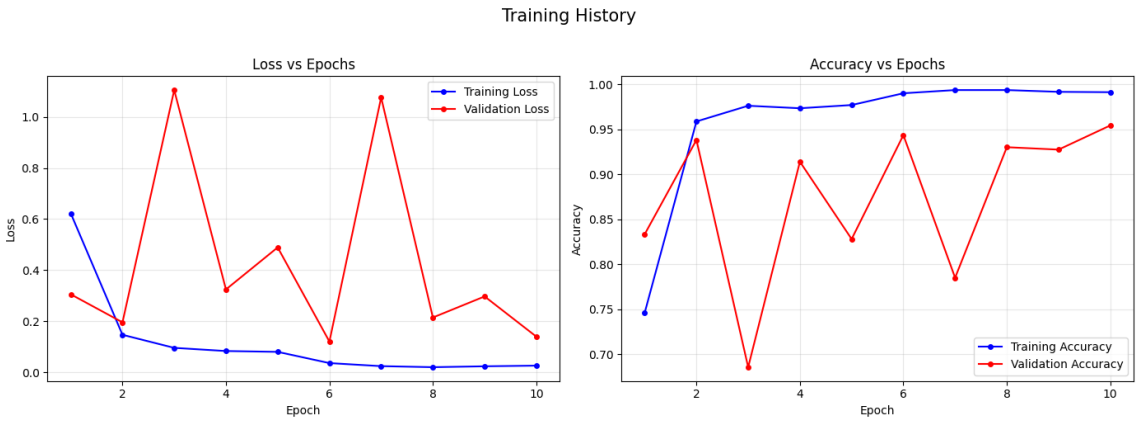

Figure 2: Training and validation loss (left) and accuracy (right) over 10 epochs.

4.2 Evaluation on the Standard Test Set

On the held-out RPS test set (372 images), the final model achieved an overall accuracy of **95%**. The per-class classification report is shown in Table 6.

Table 6: Classification report on the standard test set

Class	Precision	Recall	F1-score	Support
paper	1.00	0.86	0.93	124
rock	0.95	1.00	0.97	124
scissors	0.93	1.00	0.96	124
Accuracy		0.95		372
Macro avg	0.96	0.95	0.95	372
Weighted avg	0.96	0.95	0.95	372

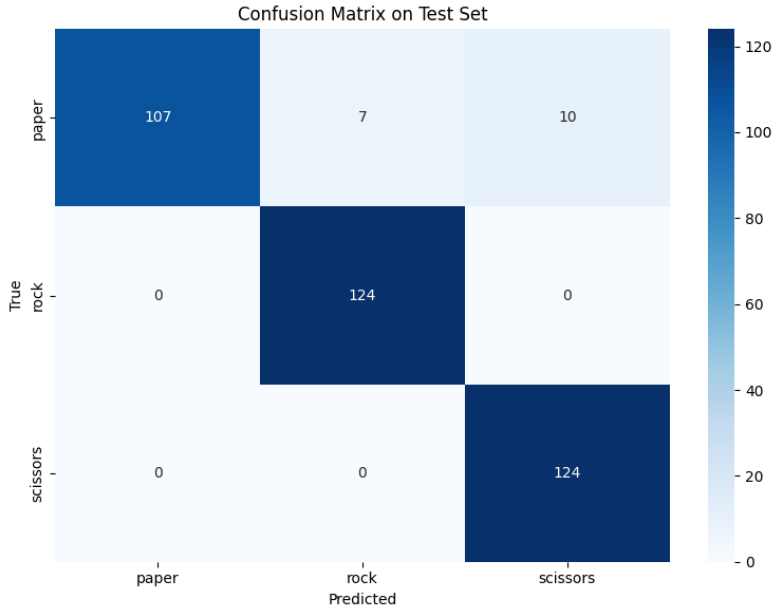

Figure 3: Confusion matrix on the standard RPS test set. `rock` and `scissors` were classified perfectly; the majority of errors (17 total) came from `paper` images being confused with `rock` or `scissors`.

The confusion matrix confirms that `rock` (124/124) and `scissors` (124/124) were classified with 100% recall, while `paper` was the weakest class (107/124 correct, recall 0.86), being confused most often with `scissors` (10 images) and occasionally with `rock` (7 images).

4.3 Visual Error Analysis

Out of 372 test images, 17 were misclassified. Three randomly sampled misclassified examples are shown in Figure 4; all three happen to be `paper` images that were confused with `scissors` or `rock`, consistent with the confusion matrix above. Visually, these particular hand poses show the fingers close together or partially curled, which likely makes the rendered silhouette ambiguous between an open `paper` hand and a partially-closed `scissors`/`rock` hand.

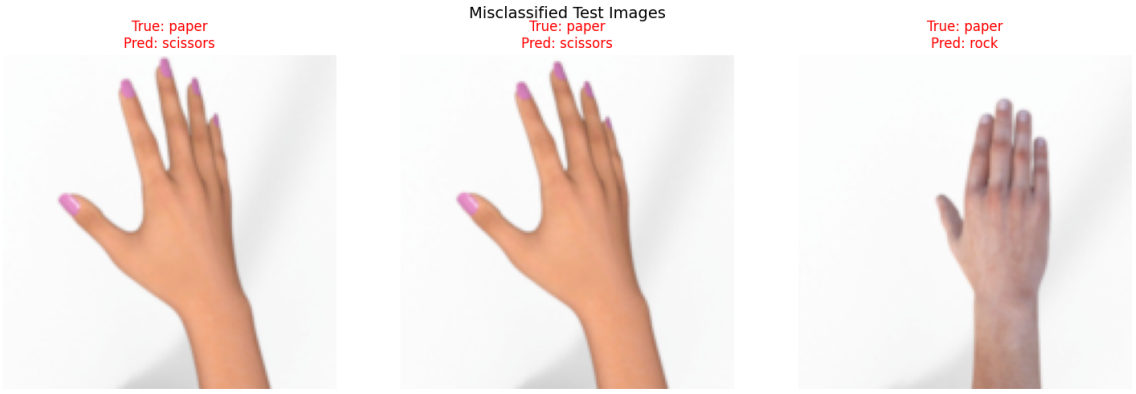

Figure 4: Three randomly selected misclassified test images with true vs. predicted labels.

4.4 Real-World Prediction on Custom Phone Images

The trained model was applied to the 10 custom smartphone photographs of the author's hand. Table 7 lists the prediction and confidence for every image, and Figure 5 shows the full prediction gallery (green title = correct, red title = incorrect).

Table 7: Model predictions on the 10 custom phone images

Filename	True Label	Predicted	Confidence	Correct?
paper_1.jpeg	paper	rock	99.9%	No
paper_2.jpeg	paper	rock	100.0%	No
paper_3.jpeg	paper	rock	97.6%	No
rock_1.jpeg	rock	rock	100.0%	Yes
rock_2.jpeg	rock	rock	100.0%	Yes
rock_3.jpeg	rock	rock	100.0%	Yes
rock_4.jpeg	rock	rock	100.0%	Yes
scissors_1.jpeg	scissors	rock	100.0%	No
scissors_2.jpeg	scissors	rock	100.0%	No
scissors_3.jpeg	scissors	rock	100.0%	No

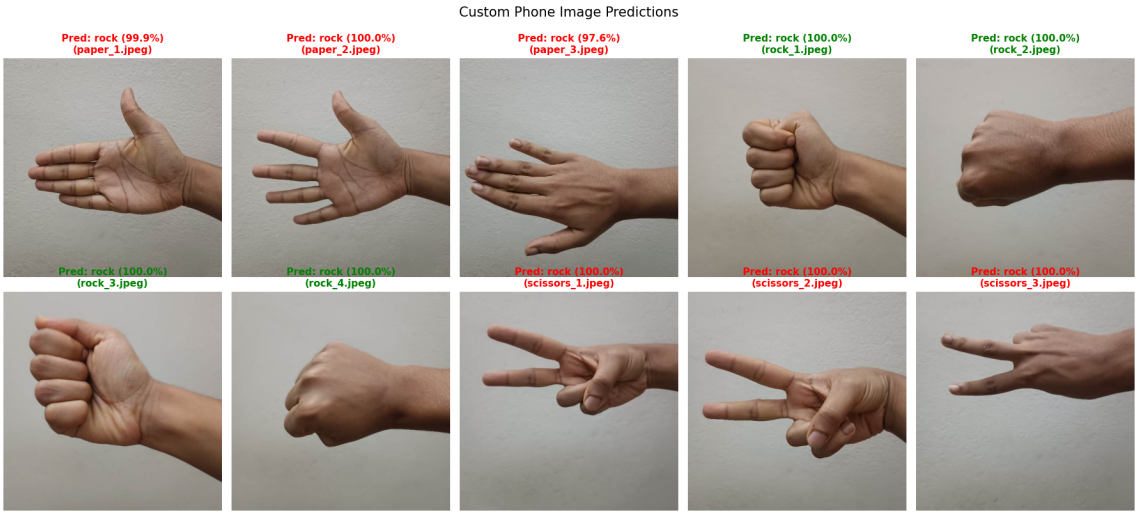

Figure 5: Predictions on all 10 custom smartphone photographs, with predicted class and confidence percentage. Green titles indicate a correct prediction; red titles indicate an incorrect one.

On the custom images, the model reached only **4/10 (40%) accuracy**, correctly identifying every `rock` image but classifying *every single* `paper` and `scissors` image as `rock` as well, almost always with near-100% confidence. This is discussed further in Section 5.

5 Discussion

5.1 Performance on the Standard Dataset

The CNN generalizes well within the distribution it was trained on: 95% test accuracy after only 10 epochs, with `rock` and `scissors` classified perfectly and `paper` being the primary source of

confusion. This is consistent with the visual similarity between an open `paper` hand at certain angles and a `scissors`/`rock` hand – the standard RPS dataset already contains this kind of pose ambiguity across classes.

5.2 Domain Gap on Real-World Photos

The sharp drop in accuracy on the custom phone photographs (95% → 40%), and the fact that the model collapsed to predicting `rock` for almost every image regardless of the true gesture, is a textbook example of **domain shift / distribution shift**. Several concrete differences between the two image domains likely explain this:

- **Background:** The standard RPS dataset uses CGI-rendered hands on a nearly uniform, plain grey/white backdrop (Figure 1), whereas the custom photos were taken against a real wall with natural shadows, texture, and lighting variation.
- **Hand appearance:** The training images are synthetic 3D renders with smooth, stylized skin, while the phone photos show a real human hand with skin texture, veins, and natural imperfections – a significant appearance gap for a model trained from scratch with no pretrained features.
- **Framing/pose:** The RPS dataset frames the hand centrally at a fairly consistent distance and rotation range ($\pm 10^\circ$ augmentation), while phone photos vary more in distance, angle, and where the hand sits in the frame.
- **Small, non-pretrained backbone:** Training a randomly initialized 619K-parameter CNN from scratch on only 2,520 images encourages the network to latch onto low-level, dataset-specific cues (background texture, rendering artifacts) rather than robust, semantic hand-shape features that would transfer to real photographs.

Interestingly, the model still correctly recognized all 4 `rock` photos. A closed fist is a visually simple, compact, high-contrast shape that seems to have been easier for the network to recognize even outside the training distribution, whereas the more detailed finger configurations of `paper` and `scissors` were not.

5.3 Possible Improvements

- **Transfer learning:** Initialize the convolutional backbone from a pretrained network (e.g., ResNet18/MobileNet on ImageNet) so the model relies on more general, transferable visual features instead of dataset-specific cues.
- **Stronger/more realistic augmentation:** Add background replacement, random cropping, and stronger color/lighting jitter during training so the model is less reliant on the plain synthetic background.
- **Domain adaptation / fine-tuning:** Fine-tune on a small number of real photographs (even a handful per class) to adapt the decision boundary to the real-world domain.
- **More diverse real training data:** Incorporate a real-photo RPS dataset (rather than only the CGI-rendered one) so the training distribution better matches the deployment distribution.

6 Repository Structure and Reproducibility

The complete project (code, custom images, and trained weights) is hosted on GitHub, following the structure required by the assignment:

```
Lab Final/
|-- dataset/                  10 custom smartphone photos
|   |-- paper_1.jpeg .. paper_3.jpeg
|   |-- rock_1.jpeg .. rock_4.jpeg
|   |-- scissors_1.jpeg .. scissors_3.jpeg
|-- model/
|   |-- 220121.pth            Saved model state dictionary
|-- 220121.ipynb              Main Colab notebook
|-- requirements.txt
|-- .gitignore
|-- README.md
```

Running the notebook end-to-end (Runtime → Run all) automatically clones the repository, downloads the RPS dataset, trains the model, saves the weights, and reproduces every figure and table in this report – no manual file uploads or path edits are required.

- **GitHub Repository:** https://github.com/Marwanthe0/CSE/tree/main/Third%20Year/3-2/Artificial%20Intelligence%20and%20ML/Lab%20Final
- **Colab Notebook:** https://colab.research.google.com/drive/1lCTyW3xMkFSMuc5Su9Zgi0NG4J7OAbLm?usp=sharing

7 Conclusion

A three-block CNN was successfully implemented, trained, and evaluated in PyTorch for Rock–Paper–Scissors hand gesture classification, achieving **95% accuracy** on the standard held-out test set after just 10 epochs. The full pipeline – automated dataset retrieval, preprocessing, training, evaluation, and real-world inference – runs end-to-end in Google Colab with a single "Run all," satisfying the assignment's reproducibility requirements.

However, evaluating the model on 10 real smartphone photographs revealed a substantial **domain gap**: accuracy dropped to 40%, with the model defaulting to the `rock` prediction for nearly every real image. This highlights an important, general lesson in applied deep learning – strong performance on a benchmark test set (drawn from the same distribution as the training data) does not guarantee robustness to real-world deployment conditions. Bridging this gap would require techniques such as transfer learning, domain-realistic augmentation, or fine-tuning on real photographs, as discussed in Section 5.